

book website:- <https://jax-ml.github.io/scaling-book/sharding/>

doc link:-

https://docs.google.com/document/d/16ogD2ipwt2AZetSoC5S4L7_hvWEAY00QrQBm6JExuqg/edit?usp=sharing

Pop Quiz [2D sharding across 1 axis]: Consider an array `fp32[1024, 4096]` with sharding `A[IXY,J]` and mesh `{'X': 8, 'Y': 2}`. How much data is held by each device? How much time would it take to load this array from HBM on H100s (assuming $3.4e12$ memory bandwidth per chip)?

My answer:-

1st index sharded across XY axis and 2nd index without sharded

so bytes per TPU= $4*4*MB/16=1MB$

time = $1000000/3.4e12=300ns$ around

Pop Quiz: Let A be an array with shape `int8[128, 2048]`, sharding `A[IXY,J]` and mesh `Mesh({'X': 2, 'Y': 8, 'Z': 2})` (so 32 devices total). How much memory does A use per device? How much total memory does A use across all devices?

My answer:-

so because Z=2 it loads whole array is both Z=0 and Z=1 mesh slice so its twice then total array size in total so the total memory = 512KB

Pop Quiz 2 [AllGather time]: Using the numbers from Part 2, how long does it take to perform the `AllGather([E,F]) → [E,F]` on a TPU v5e with a 2D mesh `{'X': 8, 'Y': 4}`, $E=2048$, $F=8192$ in `bfloat16`? What about with $E=256$, $F=256$?

My answer:-

$T_{hop} = V/n * w_{ici}$

$T_{total} = V/n * (w_{ici}) * (n-1)$

<https://www.desmos.com/calculator/5ualxkt9ei>

total for birection part 1 is 559 micro sec for uni direction

2.1 for uni direction part 2

Question 1 [replicated sharding]: An array is sharded

$A[IX, J, K, \dots]$ (i.e., only sharded across X), with a mesh `Mesh({'X': 4, 'Y': 8, 'Z': 2})`. What is the ratio of the total number of bytes taken up by A across all chips to the size of one copy of the array?

my answer:-

per TPU core size of A stored = $\text{Sizeof}(A)/|X|$

total number of TPU = $X \cdot Y \cdot Z$

therefore ratio is $Y \cdot Z = 16$

Question 2 [AllGather latency]: How long should $\text{AllGather}_X([B_X, D_Y])$ take on a TPU v4p 4x4x4 slice with mesh `Mesh({'X': 4, 'Y': 4, 'Z': 4})` if $B=1024$ and $D=4096$ in bfloat16?

How about $\text{AllGather}_{XY}([B_X, D_Y])$?

How about $\text{AllReduce}_Z([B_X, D_Y], [U_Z])$?

My answer:-

all gather across one dimension have constant time $|V|/w|C|$

part 1) 23 micro sec

part 2) 46 micro sec

part 3) 11.6 micro sec

<https://www.desmos.com/calculator/7n9ee6lsm4>

Question 3 [latency-bound AllGather]: Let's say we're performing an $\text{AllGather}_X([B_X])$ but B is very small (say 128). How long should this take on a TPU v4p 4x4x4 slice with mesh `Mesh({'X': 4, 'Y': 4, 'Z': 4})` in bfloat16? *Hint: you're probably latency bound.*

my answer 2 micro second because of overhead

<https://www.desmos.com/calculator/tl1akoexql>

Question 4 [matmul strategies]: To perform

$X[B, D] \cdot DY[DX, F] \rightarrow Z[B, F]$, in this section we tell you to perform

AllGatherX($Y[D_X, F]$) and multiply the fully replicated matrices (Case 2, Strategy 1). Instead, you could multiply the local shards like $X[B, D_X] \cdot DY[D_X, F] \rightarrow Z[B, F] \{UX\}$ (Case 3, Strategy 2), and then AllReduceX($Z[B, F] \{UX\}$) How many FLOPs and comms does each of these perform? Which is better and why?

My answer:- 2 bytes per element

part 1=

AllGatherX($Y[D_X, F]$) time = $2 \cdot D \cdot F / W_{ici}$

multiply time = $2 \cdot BDF / w_{flop}$

for $B > w_{flop} / w_{ici}$ its compute bound so strategy 1 more large batch sizes

part 2:-

multiply time = $2 \cdot BDF / X \cdot w_{flop}$

ALL reduce = $4 \cdot BF / W_{ici}$

for $D > 2 \cdot X \cdot w_{flop} / w_{ici}$ its comput bound and independent of B
strategy 2 for large models

Question 5 [minimum latency]: Let's say I want to do a matmul

$A[I, J] \cdot JB[J, K] \rightarrow C[I, K]$ on a TPU v4p 4x4x4 with the lowest possible latency.

Assume the inputs can be sharded arbitrarily but the result should be fully replicated.

How should my inputs be sharded? What is the total FLOPs and comms time?

My answer:-

ways of sharding

- 1) $A[I_{XYZ}, J] \times B[J, K] = A[I, J] \times B[J, K_{XYZ}]$ Tcomm = $I \cdot K / W_{ici_bi}$ and
Tcomp = $2 \cdot IJK / XYZ \cdot \text{flop_speed}$ floating operation (multiply partly and then gather all)
- 2) $A[I, J_{XYZ}] \times B[J, K]$ Tcomm = gather all = same as old without sharding
, Tcomp = $2 \cdot IJK / \text{flop_speed}$ (gather all then multiply) its a waste i think
- 3) $A[I, J] \times B[J, K]$ Tcomm = 0, T_comp = $2 \cdot IJK / \text{flop_speed}$

4) $A[I,J_XYZ] \times B[J_XYZ,K]$ $T_{comm} = \text{all reduce} = 2 \cdot I \cdot K / W_ici_bi$ and
 $T_{comp} = 2 \cdot IJK / XYZ \cdot \text{flop_speed}$

in my conclusion sharding against constrasting dimention of multiplication is best for computationally bound and normal for communactionaly bound

Question 6: Let's say we want to perform $A[IX,JY] \cdot JB[JY,K] \rightarrow C[IX,K]$ on TPU v5e 4x4. What communication do we perform? How much time is spent on communication vs. computation?

What about $A[IX,J] \cdot JB[JX,KY] \rightarrow C[IX,KY]$? This is the most standard setting for training where we combine data, tensor, and ZeRO sharding.

What about $A[IX,J] \cdot JB[J,KY] \rightarrow C[IX,KY]$? This is standard for inference, where we do pure tensor parallelism (+data).

My answer:-

part 1 if normal gather all 2 times $T_{comp} = 2 \cdot IJK / (XW_ici_bi)$ $T_{comm}(\text{all reduce}) = 2 \cdot IK / XW_ici_bi$

part 2: 1 gather all along J $T_{comm} = J \cdot K / YW_ici_bi$ and $T_{comp} = 2 \cdot IJK / XY \text{Flop_speed}$

part 3: its just $T_{comp} = 2 \cdot IJK / XY \text{Flop_speed}$

Question 7: A typical Transformer block has two matrices $W_{in}[D,F]$ and $W_{out}[F,D]$ where $F \gg D$. Say we have a batch size B. Then the full block is $In[B,D] \cdot W_{in}[D,F] \cdot W_{out}[F,D]$. Let's pick $D=8192$, $F=32768$ and $B=128$ and assume everything is in bfloat16. Assume we're running on a TPU v5e 2x2 slice but let's pretend each TPU only has 300MB of free memory. How should In, W_{in} , W_{out} , and Out be sharded to stay below the memory limit while minimizing overall time? How much time is spent on comms and FLOPs? *Hint: the final output doesn't need to be fully replicated, but it should be sharded the same as the input so the "layer" can be repeated.*

My answer:-

1st the sizes

in=out=2 MB

Win=Wout=500+ MB <600MB

so an itution is shard win and wout in xy

so we shard like

In[B,D_XY] Win[D,F_XY] Wout[F_XY,D]

1st Win[D,F_XY] Wout[F_XY,D]

we do multiply first

so we need to apply reduce scater to collect and shard in some direction

like W_processed[D,D]{U_XY} apply reduce scatter on 2nd dimation

reduce scatter XY,J W_processed[D,D]{U_XY}= W_collected[D,D_XY]

now we need to multiply In[B,D_XY] * W_collected[D,D_XY] so just gather all and do it

so out is [B,D_XY]

Question 8 [challenge]: Using the short code snippet above as a template, allocate a sharded array and benchmark each of the 4 main communication primitives (AllGather, AllReduce, ReduceScatter, and AllToAll) using pmap or shard_map. You will want to use `jax.lax.all_gather`, `jax.lax.psum`, `jax.lax.psum_scatter`, and `jax.lax.all_to_all`. Do you understand the semantics of these functions? How long do they take?

My answer:-

TBD

Question 10: Fun with AllToAll: In the table above, it was noted that the time to perform an AllToAll is a factor of 4 lower than the time to perform an AllGather or ReduceScatter (in the regime where we are throughput-bound). In this problem we will see where that factor of 4 comes from, and also see how this factor would change if we only had single-direction ICI links, rather than bidirectional ICI links.

1. Let's start with the single-direction case first. Imagine we have D devices in a ring topology and want to do either an AllGather or a ReduceScatter on an $N \times$

- N matrix $A[IX,J]$ (say D divides N for simplicity). Describe the comms involved in these two collectives, and calculate the total number of scalars (floats or ints) which are transferred across a single ICI link during the entirety of this algorithm.
- Now let's think about an AllToAll, still in the single-directional ICI case. How is the algorithm different in this case than the all-gather case? Calculate the number of scalars that are transferred across a single ICI link in this algorithm.
 - You should have found that the ratio between your answers to part (a) and part (b) is a nice number. Explain where this factor comes from in simple terms.
 - Now let's add bidirectional communication. How does this affect the total time needed in the all-gather case?
 - How does adding bidirectional communication affect the total time needed in the AllToAll case?
 - Now simply explain the ratio between AllGather time and AllToAll time in a bidirectional ring.

My answer:-

$D-1$ hop in single direction and the time per hop is $N^2/D \cdot W_{ici}$
 so all gather and reduce scatter time is $(D-1) N^2/D \cdot W_{ici_singledirection}$

2) num bits per hops is $((N-n/d) + (n-2n/d) + \dots + n/d)$ (for each step) $\cdot N/D$ (number of elems in each shared part in other direction) so total bits is $n^2 \cdot 2 \cdot (D-1)/2D$

3) ratio is 2 so like reduce scatter have reducing number of transferred bits in arithmetic progression which leads to average trances of bytes as half of maximum

4) for bidirection num hops is $D/2$ and per hop bits is $N^2/D \cdot 2$ so total is N^2 bytes for int array
 time is N^2/W_{ici_bi} so around twice fast

5) for all number of hops reduces by 2 in total but not only this the number of bytes also reduces by half as some bytes leave behind so it reduces by factor of 4

6) for bi direction all to all is 4 times faster than gather all

